

Extending Qwen 2.5 0.5B to Qwen 3.5 9B

Summary

This project will extend our existing Qwen 2.5 0.5B implementation so that it can load and run Qwen 3.5 9b at Q4 quantization.

Background Information

Qwen3.5 is the latest Qwen model series and uses hybrid attention. It is much more performant than Qwen 2.5 at the same sizes and is much better for agentic performance (e.g. OpenClaw and coding). I plan on using it for a Hermes agent for my dad.

The most important new computation is the Gated DeltaNet layer. Instead of storing all previous keys and values and doing full softmax attention at every layer, the linear-attention layers keep a recurrent state that is updated as tokens stream through the model. A simplified version of the update is:

```
prediction_t = state_{t-1} * key_t
delta_t = value_t - prediction_t
state_t = decay_t * state_{t-1} + step_t * outer(delta_t, key_t)
output_t = state_t * query_t
```

The exact Qwen implementation adds learned gates, short convolutions, normalization, and output projections, but this recurrence is the main reason the model can support much longer contexts without the same KV-cache growth as standard attention. The full-attention layers still need the normal KV cache, while the linear-attention layers need a persistent recurrent state.

All in all, I will implement

- loading different weight quantizations (e.g. adding q4 support with `input_float_t`)
- updating KV cache to include recurrent state.
- tiled GEMM for attention
- kernel for recurrent mechanism for gated delta net (explain this).
- sampling of tokens with temperature instead of argmax like we are doing now.

Questions to Address

There are already GPU implementations of this architecture in larger inference stacks. Hugging Face has model support, llama.cpp/GGUF support exists or is being added through Qwen-compatible implementations, and the Gated DeltaNet kernels are closely related to work in NVLabs' GatedDeltaNet repository and the Flash Linear Attention project. I do not plan to match those implementations, but I can use them as references for tensor shapes, recurrence structure, and what a more optimized kernel would look like.

The main questions I want to answer are:

- How are kernels different for the modern Blackwell chips like the H100 compared to my shitty RTX 2070 Super/GTX 1080?
- How does parallelization work for linear attention mechanisms?

The hardest parts will probably be understanding the GGUF/Q4 weight layout, getting every tensor shape right, and debugging CUDA indexing mistakes.

Deliverables and Goals

- A Python reference implementation for the Qwen 3.5 pieces that are not already in my Qwen 2.5 project.

- Single-threaded CPU kernels for the Gated DeltaNet update, mainly for debugging.
- CUDA support for loading and using Q4 quantized weights.
- A working Qwen 3.5 9B inference path for short prompts.
- A CUDA kernel for the Gated DeltaNet recurrent state update.
- Basic temperature sampling.
- Profiling results and a short final report explaining what worked, what was too slow, and what I would optimize next.

Week-by-Week Timeline

- week 1-2: pyref python implementation of Qwen3.5 + single-thread kernels
- week 3: parallelized CUDA kernels
- week 4: profiling + optimization for my specific GPU, e.g. checking tradeoffs for tiled flash attention, fusing kernels, etc...